# Homework 3

Introduction to Blockchain (ITB)

**Semester**: Spring 2026
**Instructor**: Ahmed Akhtar
**Date**: Friday, 10th April 2026

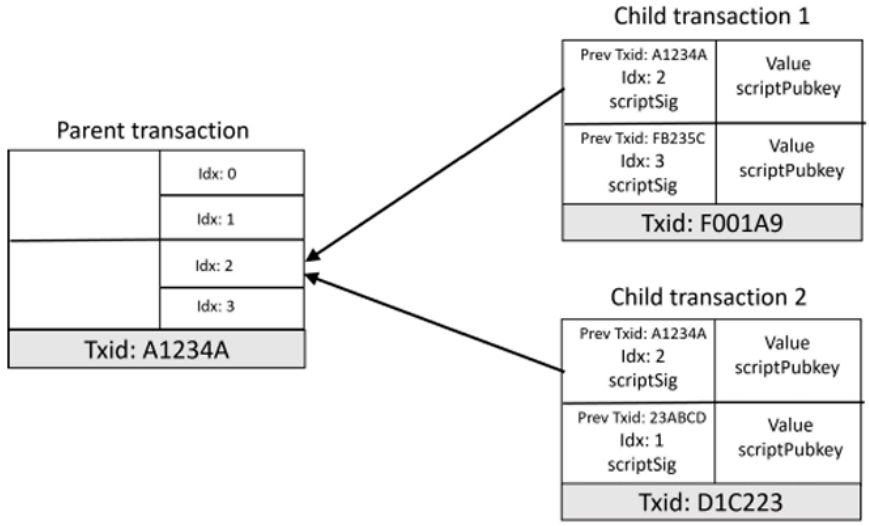
## Members


1. Izza Sohail Khan, 28996
2. Zuha Aqib, 26106


## Table of Contents

# Question 1 [7 marks]

Consider the following image:




Are both of the transactions valid? Why and why not? If any of the two transactions is invalid, explain how the UTXO model inherently prevents this problem.

No, both transactions are not valid. Child Transaction 1 (F001A9) references Prev Txid: A1234A, Idx: 2 and Prev Txid: FB235C, Idx: 3 as its two inputs. Child Transaction 2 (D1C223) references Prev Txid: A1234A, Idx: 2 and Prev Txid: 21ABC0, Idx: 1 as its two inputs. Both transactions attempt to spend A1234A:Idx2, which constitutes a double-spend. F001A9 is valid in isolation. D1C223 is invalid because it attempts to spend A1234A:Idx2, which is already consumed by F001A9, only one can ever be confirmed. This means both transactions are attempting to spend the exact same UTXO using the output at index 2 of the parent transaction A1234A, which constitutes a double-spend attempt. Child Transaction 1 (F001A9) is valid in isolation because it spends A1234A:Idx2 and FB235C:Idx3, both of which are legitimate unspent outputs. Child Transaction 2 (D1C223), however, is invalid because it attempts to spend A1234A:Idx2 which is already being consumed by F001A9; only one of the two can ever be confirmed.

The UTXO model inherently prevents this problem through its fundamental design principle: every unspent output exists as a distinct, discrete entry in the global UTXO set, and consuming it as an input permanently removes it from that set. When F001A9 is confirmed first, A1234A:Idx2 is deleted from the UTXO set immediately. Any node or miner that subsequently receives D1C223 and attempts to validate it will look up A1234A:Idx2 in the UTXO set, find that it no longer exists, and immediately reject the transaction as invalid. A UTXO can be spent exactly once. There is no concept of a "balance" that could be simultaneously decremented by two competing transactions. The model makes double-spending structurally impossible at the protocol level once any one of the conflicting transactions is confirmed.

# Question 2 [10 marks]

Explain how multi-signature (multisig) transactions are created in the Bitcoin network. How did Segwit improve the handling of multi-sig transactions? Discuss the differences in transaction size and verification process. Also explain how segwit improved on the security of multisig transactions.

A multi-signature (multisig) transaction in Bitcoin requires a minimum of m valid signatures out of n possible public keys in order to unlock and spend funds, commonly written as an m-of-n scheme. In legacy Bitcoin, multisig was implemented using P2SH (Pay-to-Script-Hash). The party locking the funds creates a redeemScript of the form OP_2 <pk1> <pk2> <pk3> OP_3 OP_CHECKMULTISIG for a 2-of-3 setup, hashes it with HASH160 to produce a 20-byte script hash, and publishes the locking script (scriptPubKey) as OP_HASH160 <redeemScriptHash> OP_EQUAL. To spend the funds, the required signatories provide an unlocking script (scriptSig) containing OP_0 <sig1> <sig2> <redeemScript>. The OP_0 is a workaround for a known off-by-one bug in OP_CHECKMULTISIG which consumes one extra stack item. All of this data which includes the signatures, public keys, and the redeemScript itself travels in the main transaction body as part of the scriptSig, which is included in the txid computation.

SegWit improved multisig handling in three key dimensions: transaction size and fees, the verification process, and security. Regarding size and fees, in legacy P2SH multisig all witness data (signatures, redeemScript) is in the scriptSig and counted at full weight toward the transaction size. In SegWit's P2WSH (Pay-to-Witness-Script-Hash), the equivalent data is moved to a separate witness field, which is discounted to one quarter of the cost of base transaction data under the block weight calculation. This means a P2WSH multisig transaction is effectively cheaper to include in a block, reducing fees for users.

Regarding the verification process, the P2WSH scriptPubKey is simply OP_0 <32-byte-hash>, making the locking script extremely compact. To spend the output, the spender provides the witnessScript and the required signatures in the witness field; the scriptSig is left empty. A SegWit-aware node recognises the OP_0 <32-byte-push> pattern as a P2WSH witness program. It extracts the last witness stack item as the witnessScript, computes SHA256(witnessScript), and checks that the result equals the 32-byte hash committed in the scriptPubKey. If it matches, the node executes the witnessScript using the remaining witness items as the initial stack. For a 2-of-3 multisig, it verifies that at least two valid ECDSA signatures are provided against the listed public keys. This contrasts with legacy P2SH, where the node must concatenate the scriptSig (containing signatures, public keys, and the redeemScript) with the scriptPubKey and execute the combined script, a more verbose and less clean process that also includes all witness data in the base transaction weight.

# Question 3 Segwit Multisig Account [24 pts]

Eman, Namwar, and Zaeem have locked 3 BTC in ‘eman-namwar-zaeem‘ joint account by creating a segwit ‘pay to witness script hash‘ (p2wsh) transaction. Assume that the txid of the locking transaction is TXABC123 and it has two outputs. The funds are locked at the first output (index 0), while a 32 B hash of an important document called ‘Project Sigma‘ is stored at the second output (index 1).

## Part (a)

a) Let denote the script hash and denote the hash of Project Sigma. Write down the complete scripts for the two outputs of this transaction. (4 pts)

The locking transaction TXABC123 has two outputs. Output 0 locks the 3 BTC in a P2WSH multisig and its scriptPubKey is:

> OP_0 <sh>
> where <sh> is the SHA256 hash of the witnessScript, and the witnessScript itself is OP_2 <pkE> <pkN> <pkZ> OP_3 OP_CHECKMULTISIG.

Output 1 stores the hash of Project Sigma on-chain using an OP_RETURN output, making it provably unspendable:

> OP_RETURN <psigh>
> where <psigh> is the 32-byte hash of the Project Sigma document.

## Part (b)

b) What will be the increase in UTXO database size once the txid TXABC123 is confirmed and added to the blockchain? (3 pts)

Only outputs that are spendable are entered into the UTXO database. The OP_RETURN output at index 1 is provably unspendable and is therefore never added to the UTXO set. Only Output 0 (the P2WSH output) is added. A single P2WSH UTXO entry in the database stores the txid (32 bytes), the output index (4 bytes), the value in satoshis (8 bytes), and the scriptPubKey which for P2WSH is OP_0 plus a 2-byte push opcode plus the 32-byte script hash totalling 34 bytes. A single P2WSH UTXO entry in the database stores the txid (32 bytes), the output index (4 bytes), the value in satoshis (8 bytes), the scriptPubKey which for P2WSH is OP_0 plus a 2-byte push opcode plus the 32-byte script hash, totalling 34 bytes, and a small metadata field encoding the block height and whether the output is from a coinbase transaction (approximately 4 bytes). This gives a total of approximately 82 bytes. The UTXO database therefore increases by approximately 82 bytes once TXABC123 is confirmed.

## Part (c)

c) What is the advantage of storing the hash of Project Sigma on the blockchain? (2 pts)

Storing the hash of Project Sigma on the blockchain provides immutable timestamping and integrity verification. Because the Bitcoin blockchain is append-only and cryptographically linked, the presence of <psigh> in a confirmed transaction proves that the document existed in its exact current form at the time the transaction was mined. Any subsequent alteration to the document, even a single bit, would produce an entirely different hash, and the mismatch with the on-chain value would be immediately detectable by anyone who rehashes the document and compares it to <psigh>. This provides a trustless, decentralised proof of existence and integrity without storing the full document on-chain, keeping the blockchain compact since only 32 bytes are committed.

# Part (d)

d) Suppose Eman and Namwar have to pay 0.8 BTC to Sana. The spending rules of txid TXABC123 require 2 of 3 signatures. Eman and Namwar create a new segwit spending txn with txid TXDEF456, which pays 0.8 BTC to Sana and locks the remaining funds in the 'eman-namwar-zaeem' joint account with the same spending rules as txid TXABC123. Write the complete scripts of the input including the witness field and the scripts of all the outputs of txid TXDEF456. Assume and represent the signatures of Eman and Namwar and , , and are the public keys of Eman, Namwar and Zaeem. The pubkey hash of Sana is . Assume the transaction fee is 0.1 BTC. (5 pts)

**Input of TXDEF456 (spending TXABC123, Output 0):**
Prev Hash: TXABC123
Index: 0
scriptSig: (empty)
Witness stack (4 items):

- Item 1: <> (empty byte vector: satisfies OP_CHECKMULTISIG off-by-one bug)
- Item 2: <sigE> (Eman's DER-encoded ECDSA signature)
- Item 3: <sigN> (Namwar's DER-encoded ECDSA signature)
- Item 4: <witnessScript> (serialised byte string whose content is: OP_2 <pkE> <pkN> <pkZ> OP_3 OP_CHECKMULTISIG)

Note: Item 4 is a serialised script pushed as a data blob onto the witness stack. Its opcodes are shown above for readability, but the witness field carries it as raw bytes, not as executed opcodes. The SegWit validator extracts this item, hashes it with SHA256, and checks the result against <sh> in the scriptPubKey before executing it.

**Output 0 of TXDEF456:** Pay 0.8 BTC to Sana (P2WPKH, preferred in a SegWit tx):
Value: 0.8 BTC
scriptPubKey: OP_0 <pkhS>
(P2PKH is also acceptable; P2WPKH is the native SegWit equivalent and keeps the transaction fully within the SegWit framework.)

**Output 1 of TXDEF456:** Lock remaining 2.1 BTC back in the joint account (P2WSH):
Value: 2.1 BTC
scriptPubKey: OP_0 <sh>

The value arithmetic is: 3 BTC input: 0.8 BTC (Sana) − 0.1 BTC (fee) = 2.1 BTC returned to the joint account

## Part (e)

e) Suppose a segwit node wants to validate txid TXDEF456. Assuming that the transaction is valid, explain all the steps involved in the validation procedure for the segwit node. (3 pts)

When a SegWit node validates TXDEF456, it first looks up the outpoint TXABC123:0 in its local UTXO set to confirm it exists and is unspent, retrieving the stored scriptPubKey OP_0 <sh> and the value of 3 BTC. It recognises the pattern OP_0 followed by a 32-byte push as a P2WSH witness program. It then extracts the last item of the witness stack which is the witnessScript OP_2 <pkE> <pkN> <pkZ> OP_3 OP_CHECKMULTISIG and computes SHA256(witnessScript), checking that the result equals <sh> from the UTXO's scriptPubKey. If the hash matches, it executes the witnessScript using the remaining witness items (<>, <sigE>, <sigN>) as the initial stack. OP_CHECKMULTISIG verifies that both <sigE> and <sigN> are valid ECDSA signatures over the transaction data using <pkE> and <pkN> respectively, and that they meet the 2-of-3 threshold. Finally, the node verifies conservation of value: inputs (3 BTC) must equal outputs (0.8 + 2.1) plus fee (0.1), which holds. If all checks pass, the node marks TXABC123:0 as spent and adds the two new UTXOs to the UTXO set.

## Part (f)

f) Suppose a non-segwit node (old node) wants to validate txid TXDEF456. Assuming that the transaction is valid, explain all the steps involved in the validation procedure for the old non-segwit node. (3 pts)

A non-SegWit (old) node validates TXDEF456 as follows. It reads the input and finds the scriptSig is empty. It retrieves the scriptPubKey of TXABC123:0 from its copy of the blockchain (not a UTXO set in the modern sense), which is OP_0 <sh>. It executes the combined script: the empty scriptSig contributes nothing, then OP_0 pushes the integer 0 onto the stack, then <sh> (32 bytes of non-zero data) is pushed onto the stack. The script terminates with a non-empty, non-zero top of stack, which under legacy script evaluation rules means the script succeeds and returns TRUE. The old node therefore accepts the transaction as valid. It never downloads or processes the witness field at all, because the witness data is in the extended serialisation format that old nodes simply ignore. The old node has effectively treated this P2WSH output as "anyone-can-spend" since it cannot enforce the witness rules but it still accepts the transaction, maintaining consensus with SegWit nodes.

## Part (g)

g) Suppose Sana knows about the transaction malleability vulnerability of Bitcoin transac- tions. She also knows a corrupt miner who can malleate transactions. Will Sana and her miner friend succeed in malleating txid TXDEF456? Explain your answer. (2 pts)

No, Sana and her corrupt miner cannot successfully malleate TXDEF456. Transaction malleability works by modifying the scriptSig in a way that keeps signatures valid but changes the txid. In a legacy transaction, this is possible because signatures reside in the scriptSig, which is part of the data hashed to produce the txid. In TXDEF456, however, the scriptSig is empty. All signature data, <sigE> and <sigN>,

resides exclusively in the witness field. The txid of a SegWit transaction is computed over only the non-witness fields: version, inputs which include outpoints and empty scriptSigs, outputs, and locktime. The witness is completely excluded from the txid computation. Therefore, modifying the witness data in any way does not change the txid, and any modification to the non-witness data would invalidate the existing signatures. SegWit was specifically designed to eliminate this attack vector, and TXDEF456 is fully protected.

# Part (h)

h) Suppose Zain knows the spending rules of ‘eman-namwar-zaeem‘ joint account and also knows all the public keys. With this information, Zain can create a serialized script whose hash equals the script hash at output 0 of txid TXABC123. Assuming that Zain is malicious, can he create a transaction to unlock the funds in the joint account? (2 pts)

No, Zain cannot unlock the funds even though he knows the witnessScript and all three public keys. The P2WSH spending rules require that whoever unlocks the output must provide not only the witnessScript which Zain can reconstruct but also at least 2 valid ECDSA signatures over the spending transaction, produced using the corresponding private keys. ECDSA is a one-way cryptographic scheme: a valid signature can only be produced by the holder of the private key corresponding to a given public key. Public keys alone cannot be used to generate valid signatures. Since Zain does not possess the private keys of Eman, Namwar, or Zaeem, he cannot produce <sigE>, <sigN>, or <sigZ>, and any transaction he constructs will fail OP_CHECKMULTISIG during validation and be rejected by the network.

# Question 4 Multiple Transactions [12 pts]

Suppose the following transactions are broadcasted simultaneously on the blockchain network and all become available to a miner.

**Transaction hash: ABC3**

| Inputs | Outputs |
|---|---|
| Some valid input worth 10 BTC | Alice → 3.95 BTC |
| | Bob → 1.95 BTC |
| | Charlie → 3.90 BTC |

**Transaction hash: BD9E**

| Inputs | Outputs |
|---|---|
| Prev hash: ABC3<br>Idx: 0<br>Signed by Alice | Dianne → 3.9 BTC |

**Transaction hash: C4B7**

| Inputs | Outputs |
|---|---|
| Prev hash: ABC3<br>Idx: 1<br>Signed by Bob | Fring → 1.90 BTC |
| Prev hash: ABC3<br>Idx: 2<br>Signed by Charlie | Bob → 1.0 BTC |
| | Charlie → 2.85 BTC |

**Transaction hash: D743**

| Inputs | Outputs |
|---|---|
| Prev hash: ABC3<br>Idx: 0<br>Signed by Alice | George → 4.0 BTC |
| Prev hash: C4B7<br>Idx: 2<br>Signed by Charlie | Hank → 2.60 BTC |

The miner has the flexibility to pick the transactions in any order. What transaction order would maximize their mining fee? What would be that fee?

The valid transactions are ABC3, BD9E, and C4B7. D743 is invalid (see Part a). Among the valid transactions, the fee-maximising order is ABC3 -> BD9E -> C4B7 (or equivalently ABC3 -> C4B7 -> BD9E, since BD9E and C4B7 are independent of each other once ABC3 is confirmed). The total fee earned is 0.35 BTC, as computed in Part (b).

## Part (a)

a) Identify all the invalid transactions (you should mention the transaction hashes, which are given). Also state a reason, why do you think a transaction is invalid (6 pts).

D743 is invalid for two independent reasons.

First, D743 references ABC3:Idx0 (Alice's 3.95 BTC output) as one of its inputs, signed by Alice. BD9E also references ABC3:Idx0, signed by Alice. Both transactions attempt to consume the same UTXO, which is a classic double-spend. Since a UTXO can only be spent once, at most one of them can be confirmed. BD9E is the valid one; D743 loses this conflict because it has no additional claim on that output.

Second, D743's other input is C4B7:Idx2. C4B7 is itself a child of ABC3 - it spends ABC3:Idx1 and ABC3:Idx2. This means D743 cannot be included in any block unless C4B7 is confirmed first, and C4B7 cannot be confirmed unless ABC3 is confirmed first. So D743 has a chain dependency on C4B7 while simultaneously double-spending ABC3:Idx0 with BD9E. Even setting aside the double-spend, a miner who tries to include D743 alongside BD9E in the same block would find it invalid because ABC3:Idx0 would already be spent by BD9E.
All other transactions such as ABC3, BD9E, and C4B7 are valid.

# Part (b)

b) Suppose the valid transactions make it to the mempool and a miner picks them up for inclusion in a block. How much fee would he receive from these transactions. Show your working (6 pts).

The valid transactions are ABC3, BD9E, and C4B7. Note that BD9E and C4B7 both depend on ABC3, so the miner must include ABC3 first.

- For ABC3: the input is 10 BTC and the outputs sum to 3.95 + 1.95 + 3.90 = 9.80 BTC, giving a fee of 0.20 BTC.
- For BD9E: the input is ABC3:Idx0 = 3.95 BTC and the output to Dianne is 3.90 BTC, giving a fee of 0.05 BTC.
- For C4B7: the inputs are ABC3:Idx1 (1.95 BTC) and ABC3:Idx2 (3.90 BTC), totalling 5.85 BTC. The outputs are Fring (1.90) + Bob (1.00) + Charlie (2.85) = 5.75 BTC, giving a fee of 0.10 BTC.

The total mining fee is 0.20 + 0.05 + 0.10 = 0.35 BTC. The optimal inclusion order is ABC3 -> BD9E -> C4B7 (or ABC3 -> C4B7 -> BD9E), since BD9E and C4B7 both require ABC3 to be confirmed first.

# Question 5 Advanced SegWit Concepts [15 pts]

Segregated Witness (SegWit) was a major upgrade to Bitcoin introduced via a soft fork in 2017. It changed how transaction data is organized and validated, enabling scalability improvements and new features.

## Part (a)

a) Explain how SegWit changes the structure of a Bitcoin transaction compared to a legacy transaction. Specifically, describe what is stored in the witness field and how it relates to the scriptSig and scriptPubKey (5 pts).

In a legacy Bitcoin transaction, the unlocking data for each input, primarily the digital signature(s) and the public key, is placed directly in the scriptSig field of the input. The scriptPubKey (locking script) sits in the output being spent. To validate a transaction, a node concatenates the scriptSig and scriptPubKey and executes the combined script. Because the scriptSig is part of the serialised transaction data, it is included in the double-SHA256 hash that computes the txid.

SegWit fundamentally restructures this. The scriptSig for a native SegWit input is empty (or, in the P2SH-wrapped variant, contains only a push of a small redeem script). The scriptPubKey becomes a compact witness program for P2WPKH it is OP_0 <20-byte-pubkeyhash> and for P2WSH it is OP_0 <32-byte-scripthash>. The signatures, public keys, and (for P2WSH) the full witnessScript are moved into a new per-input data area called the witness field, which is serialised separately from the main transaction body. The txid is computed only over the non-witness data (version, inputs with empty scriptSigs, outputs, locktime), while a separate identifier called the wtxid covers the full serialisation including the witness. SegWit transactions signal their format with a marker byte 0x00 and flag byte 0x01 inserted after the version field. The relationship between the three fields is therefore: the scriptPubKey commits to a hash of the spending conditions, the witness field reveals those conditions and provides the satisfying data (signatures), and the scriptSig is empty, its role has been entirely taken over by the witness.

## Part (b)

b) Before SegWit, Bitcoin transactions were vulnerable to a form of transaction malleability. Explain what transaction malleability is, how it could be exploited, and how SegWit solves this problem. (5 pts)

Transaction malleability is the ability to alter the txid of a valid, unconfirmed Bitcoin transaction without invalidating its signatures. In legacy transactions, the txid is the hash of the entire serialised transaction including the scriptSig. ECDSA signatures encoded in DER format have some flexibility, for example, extra zero padding bytes can be added to the S value and the signature remains valid. A third party (or even the original sender) could take a valid transaction, re-encode the signature in a slightly different but still valid DER form, and rebroadcast it. The resulting transaction would have the same economic effect where there is same inputs, same outputs, same amounts but a different txid.

This could be exploited in several ways. If Alice sends Bob a payment and Bob's system is tracking the original txid, a malleated version confirmed instead would leave Bob's system believing the payment

never arrived (since the tracked txid never confirmed), potentially causing Bob to request a duplicate payment. This vulnerability was a contributing factor in the collapse of the Mt. Gox exchange, whose withdrawal system tracked txids and was fooled into reissuing payments. It also prevented reliable construction of multi-step off-chain protocols (like payment channels) that depend on referencing a parent transaction by txid before it is confirmed.

SegWit eliminates this by removing all signature data from the inputs entirely. Since the witness field is excluded from the txid computation, no manipulation of signatures however subtle can change the txid. The txid is now a function only of the economic content of the transaction (who is spending what and paying whom), making it immutable once broadcast. Any attempt to malleate the witness simply results in a different wtxid but the same txid, and since the signatures would then fail verification (the signed message digest would no longer match), the malleated transaction would be rejected by all nodes.

## Part (c)

c) SegWit was deployed as a soft fork rather than a hard fork. Explain how this was achieved in terms of transaction compatibility. In your answer, describe how older nodes (that have not upgraded) interpret SegWit transactions and why consensus is still main- tained. (5 pts)

SegWit was deployed as a soft fork, meaning it tightened the rules rather than changing them in a way that would cause old nodes to reject new blocks. The mechanism relies on the fact that SegWit output scriptPubKeys are valid under old script rules. A P2WPKH scriptPubKey OP_0 <20-byte-hash> is a perfectly valid legacy script: OP_0 pushes zero onto the stack, then the hash bytes are pushed as data, leaving a non-empty top of stack which evaluates as TRUE (success) under legacy rules. Similarly, the spending inputs have empty scriptSigs, so the combined execution of empty scriptSig plus OP_0 <hash> succeeds trivially. Old nodes therefore see SegWit outputs as "anyone-can-spend" and accept any transaction that claims them, without ever looking at the witness data (which they don't even download in the base serialisation format).

New (upgraded) nodes, by contrast, recognise the witness program pattern and enforce the full SegWit validation rules including signature verification via the witness field. Consensus is maintained because old nodes never reject valid SegWit transactions since they evaluate them as anyone-can-spend, they always accept them. A block containing an invalid SegWit transaction (where the witness fails verification) would be accepted by old nodes but rejected by new nodes. Since the majority of mining hash rate upgraded, the longest chain is always one that satisfies both old and new rules. Old nodes follow the longest valid chain (by their weaker rules), which happens to be the same chain that satisfies SegWit rules. There is therefore no chain split: old nodes remain on the same canonical chain as new nodes, just enforcing a subset of the current rules.

# Question 6 Merkle Tree and Hashing [11 pts]

A Bitcoin block contains 2048 transactions. A lightweight SPV (Simplified Payment Verification) client wants to verify that a specific transaction is included in this block.

## Part (a)

a) Calculate the exact number of hashes the SPV client needs to receive as proof to verify the transaction's inclusion in the block. Show your calculation. [3 pts]

In a Merkle tree, all transactions are leaves and parent nodes are computed by hashing pairs of child nodes. For a block with 2048 transactions, the tree has a height of $\log_2(2048) = \log_2(2^{11}) = 11$ levels. To verify that a specific transaction is included, an SPV client reconstructs the path from the transaction's leaf to the Merkle root. At each level, it needs the hash of the sibling node it cannot compute itself. Since there are 11 levels (excluding the leaf itself), the SPV client needs 11 hashes as its Merkle proof.

## Part (b)

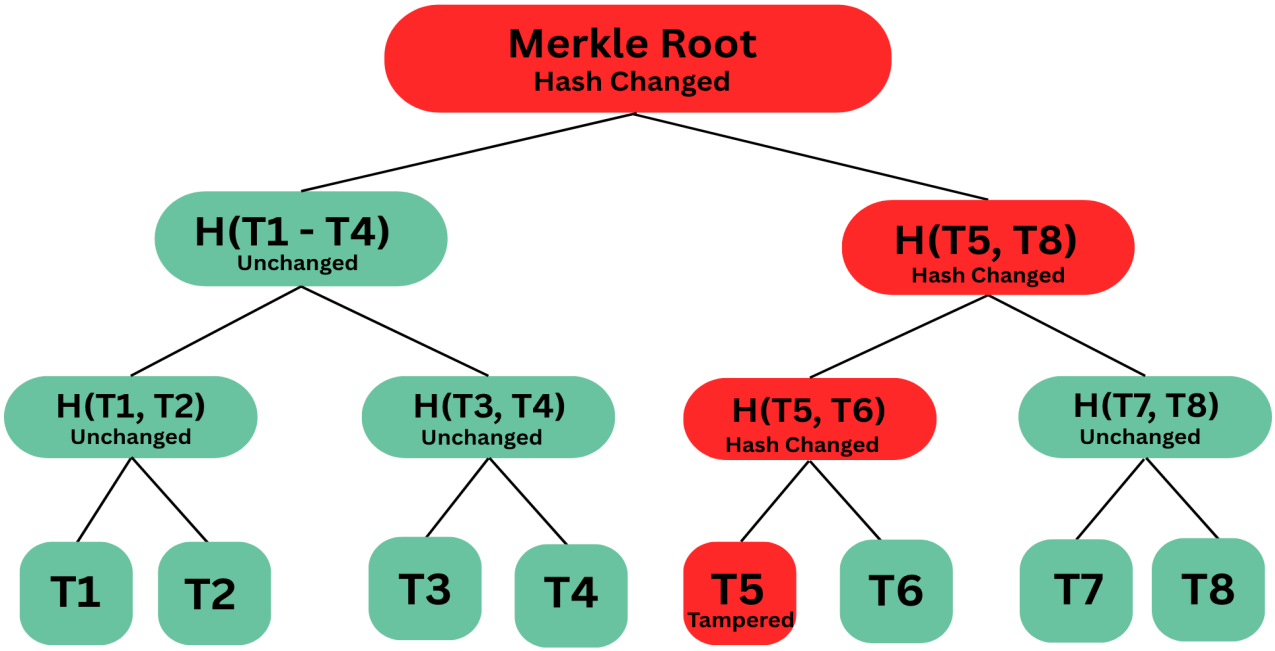
b) If the block size increases to contain 16,384 transactions, how many additional hashes would be needed for the Merkle proof compared to part (a)? Explain why Merkle trees provide logarithmic scalability. [4 pts]

For a block with 16,384 transactions, the tree height is $\log_2(16{,}384) = \log_2(2^{14}) = 14$ levels, so the proof requires 14 hashes. Compared to the 11 hashes needed for 2,048 transactions, the number of additional hashes is 14 - 11 = 3.

Merkle trees provide logarithmic scalability because the tree is binary, each level doubles the number of leaves it can cover. To verify any single leaf in a tree of n leaves, the proof consists of exactly one sibling hash per level, and the number of levels is $\log_2(n)$. This means that as the number of transactions grows exponentially (doubling each time adds only one hash to the proof length), verification complexity grows only logarithmically. Even in a block containing one million transactions, an SPV client would need only 20 hashes for its proof. This is what makes SPV lightweight clients practical on mobile devices and embedded systems without requiring the full blockchain.

## Part (c)

c) Suppose an attacker tries to create a fake Merkle proof by modifying transaction Tx5 in a block containing 8 transactions. Draw the Merkle tree and mark with an 'X' all the nodes that would have different hash values due to this modification. Explain why this makes fraud detection efficient. [4 pts]




The Merkle tree for 8 transactions has the following structure, with the four nodes affected by modifying Tx5 marked with red colour..

The modification propagates as follows: Tx5's leaf hash changes first. Its parent H(T5,T6) depends on H(Tx5) so it also changes. That node's parent H(H(T5,T6), H(T7,T8)) changes next. Finally the root, which depends on all level-1 nodes, changes. The four affected nodes are Tx5, H(T5,T6), H(H(T5,T6),H(T7,T8)), and the Merkle Root. The sibling nodes H(T6), H(T7,T8), and H(H(T1,T2),H(T3,T4)) are the three hashes an SPV client needs to recompute and verify the path, if Tx5 was tampered with, the recomputed root will not match the block header's stored Merkle root, proving fraud with just 3 hash comparisons regardless of block size.

# Question 7 UTXO Model [12 pts]

Consider the following scenario involving Alice, Bob, and Charlie:

- UTXO1: Alice owns 5 BTC at (Txid: XYZ789, Index: 0)
- UTXO2: Alice owns 3 BTC at (Txid: ABC456, Index: 1)
- UTXO3: Bob owns 2 BTC at (Txid: DEF123, Index: 0)

## Part (a)

a) Alice wants to pay Bob 6 BTC and Charlie 1.5 BTC in a single transaction. Design a complete valid transaction showing all inputs, outputs, and transaction fee. Explain why Alice needs to use both UTXOs and what happens to remaining funds. [3 pts]

Alice needs to pay a total of 6 + 1.5 = 7.5 BTC, but neither of her UTXOs alone is sufficient - UTXO1 holds only 5 BTC and UTXO2 holds only 3 BTC. In the UTXO model, each coin is a discrete, indivisible unit that must be consumed in its entirety as an input; you cannot partially spend a UTXO. Alice must therefore consume both UTXOs to get the required funds. Her transaction has two inputs (XYZ789:0 contributing 5 BTC and ABC456:1 contributing 3 BTC, totalling 8 BTC) and the following outputs: Bob receives 6 BTC, Charlie receives 1.5 BTC, and the remaining 0.5 BTC is paid to the miner as a transaction fee (or Alice could send it back to herself as change and reduce the fee). Since 8 - 7.5 = 0.5 BTC, the fee of 0.5 BTC is implicitly consumed without a change output.

## Part (b)

b) After the transaction in part (a) is confirmed, list all UTXOs added to and removed from the UTXO database. [2 pts]

The two inputs: UTXO1 at (XYZ789, Index 0) and UTXO2 at (ABC456, Index 1) are removed from the UTXO database upon confirmation, as they are now spent. Two new entries are added: Bob's 6 BTC output at (NewTxId, Index 0) and Charlie's 1.5 BTC output at (NewTxId, Index 1). Note that UTXO3 (Bob's pre-existing 2 BTC at DEF123:0) is unaffected by this transaction.

## Part (c)

c) Bob now wants to create two simultaneous transactions: T1 sending 3 BTC to Charlie, and T2 sending 3 BTC to Alice. Both transactions reference Bob’s UTXO from Alice’s payment in part (a), which contains 6 BTC at (NEW001, 0). Are both transactions valid or invalid (explain), what happens when they are broadcast to the network, and which transaction gets confirmed (what are the determining factors behind a transaction getting confirmed). [5 pts]

Both T1 and T2 individually appear valid in isolation, each references (NEW001, 0) which holds 6 BTC and tries to pay out 3 BTC, leaving 3 BTC as a fee. It is worth noting that in both T1 and T2, the full 6 BTC UTXO is consumed but only 3 BTC is assigned to an output, meaning the remaining 3 BTC is implicitly paid to the miner as a transaction fee. This unusually large fee (relative to the payment amount) would strongly incentivise any miner who receives either transaction to include it in their next block as quickly as possible, effectively accelerating the double-spend resolution in favour of whichever

transaction the miner sees first. However, they are collectively invalid as a pair because they both attempt to spend the exact same UTXO: (NEW001, Index 0). This is a classic double-spend attempt, and at most one of them can ever be confirmed.

When they are broadcast to the network, each node will accept whichever transaction it receives first and add it to its mempool, treating (NEW001, 0) as "pending spend." When the second transaction arrives referencing the same outpoint, the node will see the UTXO already claimed and reject it. Different nodes may therefore hold different transactions in their mempools, creating a temporary inconsistency across the network.

The determining factors for which transaction gets confirmed are primarily the transaction fee offered (miners rationally prioritise higher-fee transactions), miner policy regarding first-seen rules (most miners will mine whichever valid transaction arrived first in their mempool), and ultimately which transaction appears in the first block mined and accepted by the network. Once a miner includes either T1 or T2 in a confirmed block, (NEW001, 0) is permanently spent, and the other transaction becomes permanently invalid and is dropped from all mempools.

## Part (d)

d) Explain why the UTXO model prevents double-spending better than an account-based model. [2pts]

The UTXO model prevents double-spending more robustly than an account-based model for two fundamental reasons. First, each UTXO is a discrete, uniquely identifiable object (identified by txid and output index) that exists in the UTXO set exactly once. Spending it removes it entirely and atomically. There is no shared mutable balance that two concurrent transactions could both decrement before either is confirmed, there is only a binary state of "exists" or "doesn't exist." Second, validation is stateless and parallel: checking whether a UTXO is spendable requires only a lookup in the UTXO set, with no dependency on global account state. In an account-based model, two transactions spending from the same account might both appear valid when checked independently against the pre-transaction balance, and only a globally ordered execution would prevent both from succeeding. The UTXO model eliminates this race condition structurally rather than relying on execution ordering.

# Question 8 Merkle Trees and UTXO [13 pts]

Bitcoin's design uses Merkle trees in block headers to commit to transactions, while nodes maintain a separate UTXO database.

## Part (a)

a) Explain why Bitcoin uses two separate data structures (Merkle tree of transactions in blocks and UTXO database) rather than just storing all unspent outputs in a Merkle tree in the block header. Discuss at least two practical reasons related to efficiency and functionality. [5 pts]

Bitcoin uses two separate data structures because they serve fundamentally different purposes, and conflating them would sacrifice efficiency or functionality in both dimensions.

The first reason is validation efficiency. The Merkle tree in the block header is a commitment structure - it allows anyone to prove that a specific transaction was included in a specific block using a logarithmic-length proof, without downloading the entire block. However, it cannot answer the question "is this output currently unspent?" To answer that, a node would have to scan every block from the transaction's block forward to the current chain tip, looking for any subsequent spending transaction meaning an O(n) scan over potentially hundreds of gigabytes of data. The UTXO database solves this by maintaining only the current set of unspent outputs in an indexed, rapidly-queryable form. When a node validates a new transaction, it simply looks up each input outpoint in the UTXO set (an O(log n) or O(1) operation) rather than scanning the blockchain. This is what makes full node validation fast enough to process thousands of transactions per block in real time.

The second reason is storage scalability. The full blockchain records every transaction ever made and grows permanently, it currently exceeds 600 GB. If all unspent outputs had to be committed in the Merkle tree of every block header, the header would need to encode the entire UTXO set (currently around 5–10 GB) in every single block. This would make block headers enormous, destroy the efficiency of SPV proofs (whose power comes from the header being just 80 bytes), and create massive redundancy since most UTXOs don't change between blocks. The separation allows the block header Merkle tree to commit only to the transactions in that specific block (a compact, non-redundant commitment), while the UTXO database is maintained separately as a compact, mutable index that is updated incrementally with each new block.

## Part (b)

b) An SPV client receives a Merkle proof showing that transaction TxK is in Block 800,000. However, this SPV client later wants to verify that TxK hasn't been spent yet. Explain why the Merkle proof alone is insufficient for this verification, and what additional information the client would need. What security assumption must the SPV client make? [8 pts]

The Merkle proof alone is fundamentally insufficient to verify that TxK has not been spent, for a simple but important reason: a Merkle proof only proves inclusion of a transaction in a particular block at block height 800,000. It says nothing whatsoever about what happened after that block. The outputs of TxK

may have been spent in block 800,001, block 900,000, or any block up to the current chain tip. The Merkle proof has no mechanism for capturing this information, it is a static snapshot of one block's contents, not a dynamic record of output state.

To verify that TxK's outputs are currently unspent, the SPV client would need additional information from a trusted source. Specifically, it would need to query a full node (or a set of full nodes) and ask whether the outputs of TxK appear in any subsequent transaction's inputs, essentially asking the full node to consult its UTXO set. The full node would either confirm the outputs are still in the UTXO set (unspent) or indicate they have been spent and provide the txid of the spending transaction (which the SPV client could optionally verify via another Merkle proof in the block where the spending transaction was confirmed).

The critical security assumption the SPV client must make is the honest majority assumption (also called the honest miner assumption): it must trust that the majority of the network's mining hash rate is honest and that the chain it is following represents the true, valid chain. An SPV client does not validate full blocks, it only checks that block headers form a valid proof-of-work chain and that its Merkle proofs connect to those headers. A coalition of dishonest miners with sufficient hash rate could theoretically present the SPV client with a fraudulent chain where a double-spend was confirmed, and the SPV client has no cryptographic means to detect this without downloading and validating full block data. The SPV client's security therefore degrades gracefully with the assumption that the longest chain is also the honest chain, an assumption that holds in practice given Bitcoin's mining economics, but is weaker than the security guarantee enjoyed by full nodes.

In practice, SPV clients cannot directly query a full node's UTXO set over the standard Bitcoin P2P protocol. Instead they rely on one of two mechanisms: BIP37 bloom filters, where the client sends a probabilistic filter of its addresses to a full node and the node sends back relevant transactions including spending transactions; or dedicated Electrum-style servers, which maintain indexed UTXO databases and allow clients to query the unspent status of specific outputs directly. Both approaches require trusting the server or node being queried to report honestly, which reinforces rather than eliminates the honest-majority security assumption the SPV client must already make.